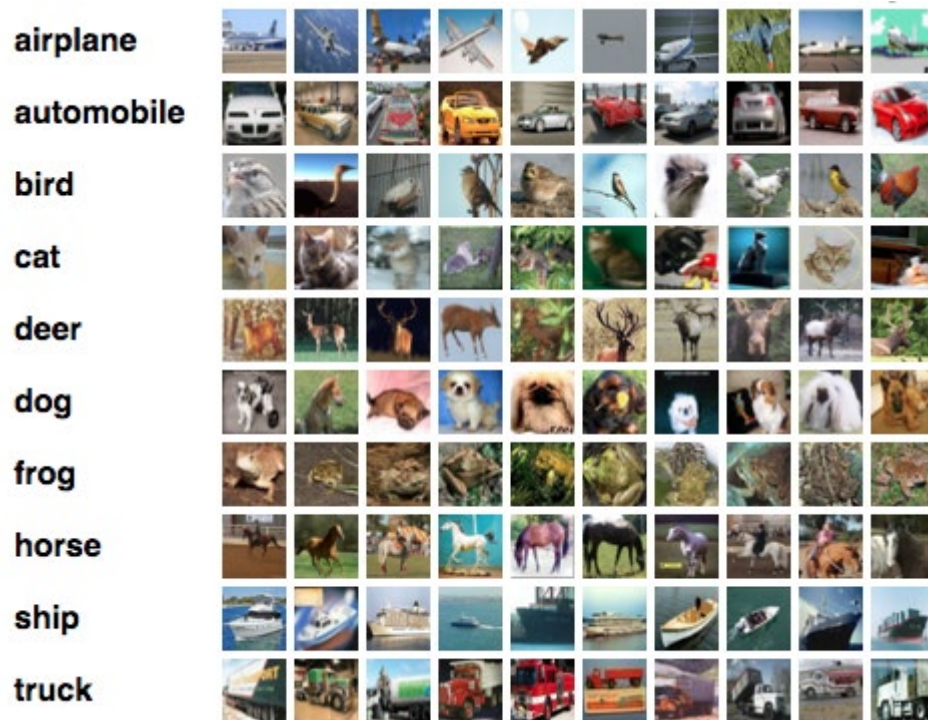


### Assignment 3: Image Classification using Convolutional Neural Networks (CNNs)

**Aim:** The Convolutional Neural Network (CNN) is a class of deep learning neural networks which represents a huge breakthrough in image recognition. They are commonly used to analyze visual imagery and are frequently used for image classification. In this assignment, your task is to learn Convolutional Neural Network (CNN) based image classifier for the CIFAR-10 dataset of images. You have to experiment with different architectures, hyper-parameters, and training procedures to make yourself familiar with training and tuning CNN models. Your aim is to obtain a final CNN architecture prevented from overfitting on CIFAR.

Reference:

CIFAR10 Dataset: Learning Multiple Layers of Features from Tiny Images, Alex Krizhevsky, 2009. Images collected by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton of the Canadian Institute For Advanced Research, available at: <https://www.cs.toronto.edu/~kriz/cifar.html>



The CIFAR-10 (Canadian Institute For Advanced Research) dataset consists of 60,000 32 x 32 colour images in 10 classes, with 6,000 images per class. There are 50,000 training images and 10,000 test images. The important points that distinguish this dataset from MNIST are:

- Images are colored in CIFAR-10 as compared to the black and white texture of MNIST.
- Each image is 32 x 32 pixel.
- 50,000 training images and 10,000 testing images.

Now, these images are taken in varying lighting conditions and at different angles, and since these are colored images, you will see that there are many variations in the color itself of similar objects (for example, the color of ocean water). In this assignment, the CIFAR-10 is used as the dataset for your learning and practice of your hyper-parameter tuning skills for CNNs. CIFAR-10 is also easily available in Keras. In Colab, you can simply load the dataset using the following code: <http://www.cs.toronto.edu/~kriz/cifar.html>

```
from keras.datasets import cifar10
# loading the dataset
(X_train, y_train), (X_test, y_test) = cifar10.load_data()
```

In jupyter notebook, the above command may not work and the procedure for getting the data is shown in a demo file.

Note: You have to use Google Colab or jupyter notebook to work on this assignment. A demo file is provided to give you some help on the initial stage.

([Assignment3\\_b\\_code\\_student.ipynb](#))

The dataset of CIFAR-10 can be obtained by clicking the following link:

<https://drive.google.com/file/d/1KZnjw6a-9maMgCffTaPSD52BxBN8fVFp/view?usp=sharing>

[cifar-10-batches-py.zip](#) (159M)

The usage is demonstrated in [Assignment3\\_b\\_code\\_student.ipynb](#)

## Your Tasks

### 1. Develop CNN\_A:

- baseline model, plain CNN
- 0.2 – 0.7 M parameters
- no use of any regularization methods such as weight decay, batch normalization, dropout etc.

### 2. Develop CNN\_B:

- wide and deep model, CNN that may have overfitting
- 10 – 15 M parameters
- no use of any regularization methods such as weight decay, batch normalization, dropout etc.

### 3. Develop CNN\_C:

- final model, CNN without/with minimal overfitting
- 1 – 5 M parameters
- less dense structure and less filters than the CNN in task2
- use of regularization methods such as weight decay, batch normalization, dropout.

Notes:

- **No need to use data augmentation or label smoothing.**
- Use ReLU activation and max-pooling layer.
- Dataset is split into train, validation and test set. Hence, you can use 10% of your 50,000 training images as validation set.
- Normalization of the training data

For each model, report:

1. Training accuracy and loss.
2. Validation accuracy and loss.
3. Final test accuracy.
4. Mean and standard deviation over three random seeds.
5. Number of parameters and computational time/cost.

Construct tables and graphs to summarize or document your results.

There can be many hyper-parameters chosen for the training/optimization, and you should choose your strategies such as:

- filter size e.g. 3x3, 5x5
- number of filters : e.g. 16, 32, 64
- max pooling size: 2x2, 3x3
- dense/hidden neuron e.g. 50, 100, 150
- batch size during training
- optimizer e.g. "sgd", "adam", "rmsprop", "nadam"
- dropout : 0.2, 0.3, 0.5 or none
- batch normalization

Show the number of parameters by `model.summary()`. State the training epochs.

## What to Submit ?

1. A pdf or WORD file of your work. Maximum SIX pages.
2. Your implementation in one .ipynb file

Page 1 : Assignment number, your **student name in English, email address,**

**10-digit student number and date of report.**

An abstract of around 300 **words** with results stated clearly. **No figure on this page.**

Page 2 : Task 1 results

A pictorial figure that gives a **visual presentation** of your CNN\_A model.

Page 3 : Task 2 results

A pictorial figure that gives a **visual presentation** of your CNN\_B model.

Page 4 and 5 : Task 3 results

A pictorial figure that gives a **visual presentation** of your CNN\_C model. Also gives your calculation of the model parameters that matches with the `model.summary()` of CNN\_C.

Page 6 : Discussions and Conclusions of your work. I expect a table that summarizes the three models, but no figure needed. Describe your work and the things learned in good English.